

CSCE 636 DEEP LEARNING · MODULE 07 · LECTURE NOTES

Transformers

The architecture that absorbed every domain.

MODULE OUTCOMES

- Explain every component of a transformer block.
- Explain multi-head attention and what the heads do.
- Explain positional encoding and compare the schemes.
- Distinguish encoder, decoder, and encoder–decoder uses.
- Explain scaling laws and what they predict.

1 The block

```
x = x + MultiHeadAttention(LayerNorm(x))    # mix ACROSS positions
x = x + FeedForward(LayerNorm(x))          # process EACH position
```

FOUR COMPONENTS, FOUR JOBS

1. MULTI-HEAD ATTENTION -- the ONLY operation that moves information between positions. Everything else is per-token.
2. FEED-FORWARD -- two linear layers and a nonlinearity, applied identically at every position, inner dimension typically 4× the model dimension. HOLDS MOST OF THE PARAMETERS. Attention mixes; the feed-forward computes.
3. RESIDUAL CONNECTIONS -- the gradient highway (M02 section 3). Without them deep transformers do not train.
4. LAYER NORM -- batch-independent, so batch size and sequence length do not affect it (M04 section 3).

PRE-NORM (as written) trains far more stably than POST-NORM (the original paper's order), which needed careful warmup to train at all.

Attention mixes across positions and the feed-forward does the computation — that division is the clearest way to read the block, and it explains the parameter distribution: the feed-forward's 4× inner dimension means it holds roughly two-thirds of a transformer's parameters, which surprises people who think of the transformer as 'the attention architecture'.

2 Heads and positions

MULTIPLE HEADS ATTEND TO DIFFERENT THINGS

Split the model dimension into h heads, run attention independently within each, and concatenate the results. The total compute is essentially unchanged; what changes is the structure.

A single attention distribution can express only one notion of relevance per position. Several heads can simultaneously attend by syntactic dependency, by coreference, by relative position, and by topic — which one softmax cannot.

And interpretable heads genuinely do appear. Previous-token heads, induction heads that complete a repeated pattern (and which appear to underlie in-context learning), and heads tracking syntactic relations have all been identified in trained models.

Though many heads are redundant. A substantial fraction can be pruned with little loss of performance — a finding replicated across models and the basis of several compression methods (Module 13). **The architecture is over-provisioned, and that appears to help training rather than inference,** which is a recurring theme.

ATTENTION HAS NO IDEA WHAT ORDER THINGS ARE IN

Self-attention is permutation-equivariant. Shuffle the input tokens and the outputs shuffle identically — **the operation sees a set of vectors, not a sequence.** This is easy to verify and genuinely surprising the first time.

So positional information must be injected explicitly, or the model cannot distinguish 'dog bites man' from 'man bites dog'. Position is not a detail of the architecture; it is a missing ingredient that must be supplied.

The schemes differ mainly in how they extrapolate beyond the lengths seen in training, which is the practically important question — a model trained on 2,000 tokens that must handle 8,000 needs an encoding whose meaning continues to make sense out there.

RoPE rotates the query and key vectors by an angle proportional to position, so that the dot product between a query at position i and a key at position j depends on $(i - j)$ — **relative position emerges from the geometry rather than being added as a term. It is the current default** and extrapolates considerably better than learned absolute encodings, which have no value at all for positions never seen in training.

Scheme	How it works	Extrapolates?
Sinusoidal	Fixed sine and cosine functions at a range of frequencies, added to the embeddings.	Somewhat. The original choice, and the frequencies were designed so that relative offsets are expressible as linear functions.
Learned absolute	A separate learned embedding vector per position index.	No. There is no embedding for a position never seen, so the model simply fails past its trained length — which is why long-context work abandoned it.

Scheme	How it works	Extrapolates?
Relative position bias	A learned bias added to the attention score, indexed by the difference between positions.	Better, and it generalises to unseen distances if the bias is bucketed.
RoPE (rotary)	Rotate Q and K by an angle proportional to position , so relative position falls out of the dot product.	Well, and it is the current default across most large models.
ALiBi	Subtract a penalty proportional to distance from the attention score. No embeddings at all.	Very well, and it is trivially simple — a striking ratio of effectiveness to complexity.

3 Three configurations

Configuration	Attention pattern	Used for
Encoder only	Bidirectional — every position attends to every other, past and future.	Classification, sequence labelling, and embeddings. BERT and its descendants. Cannot generate, because every position sees the answer.
Decoder only	Causal — masked so that each position attends only to itself and earlier positions.	Generation, and essentially all modern large language models. The mask lets every position be trained as a prediction target simultaneously, which is enormously efficient.
Encoder–decoder	Bidirectional encoder; causal decoder that additionally cross-attends to the encoder output.	Translation and summarisation , where the input and output are distinct sequences. T5, and the original transformer.

The causal mask is the entire difference between the first two — a single upper-triangular mask applied to the attention scores before the softmax. That such a small change produces models with such different characters and uses is worth sitting with, and it is a good illustration of how much of an architecture's behaviour comes from the training objective rather than the parameter arrangement.

4 Scaling

SCALING LAWS ARE SMOOTH POWER LAWS, AND THEY HELD

Loss falls as a power law in the number of parameters, the quantity of data, and the compute budget — over many orders of magnitude, with remarkably little scatter around the fitted line.

So performance became *predictable before training*, which changed the character of the work: a laboratory could fit the curve on small models, extrapolate, and commit a large budget with a forecast rather than a hope. Model development became a budgeting exercise.

Chinchilla corrected the ratio. The earlier large models were badly under-trained on data — too many parameters for the number of tokens they saw — and **parameters and training tokens should scale together**, at roughly twenty tokens per parameter for compute-optimal training. A smaller model trained on more data beat a larger one trained on less, at equal compute.

And the laws describe the *loss*, not capability. Whether a specific ability appears smoothly or suddenly depends heavily on how it is measured — a metric that is discontinuous (exact string match, multiple-choice accuracy) manufactures a discontinuous-looking curve from an underlying smooth improvement. Several reported 'emergent abilities' turn out to be artifacts of the metric, which is a measurement lesson (CSCE 633 Module 12) rather than a fact about models.

WHY THE TRANSFORMER ABSORBED EVERYTHING

A weak inductive bias (Module 06 §4) — so applying it to a new modality imposes nothing *wrong*, which is the failure mode of a strong prior in the wrong place (Module 01 §2).

An excellent hardware fit. The computation is dominated by large dense matrix multiplications, which is precisely what GPUs and accelerators are built for (CSCE 735 Module 08) — and unlike recurrence, it parallelises across the sequence.

Predictable scaling — you can plan and budget a model before building it, which no previous architecture offered.

And one implementation serves every modality. The engineering investment compounds rather than fragmenting: every kernel optimisation, every distributed training improvement, every inference trick benefits text, vision, audio, and protein models simultaneously. **That last point is underrated and may matter most** — architectural convergence meant the entire field's systems work pointed in one direction, and the compounding has been substantial.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Vaswani et al. — Attention Is All You Need (free)	The original. Short, and the ablation table in section 6 is where the design justifications are.
Karpathy — nanoGPT and 'Let's build GPT' (free)	A complete, readable, trainable implementation. Read it alongside §1 — it is about 300 lines.
Hoffmann et al. — Training Compute-Optimal Language Models (Chinchilla) (free)	The §4 correction , with the parameter-to-token ratio derived from experiments.
Schaeffer, Miranda & Koyejo — Are Emergent Abilities of Large Language Models a Mirage? (free)	The metric argument of §4 — a careful demonstration that discontinuous metrics manufacture discontinuous curves.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement a transformer block from scratch** and train a small language model on a text corpus.
- 2 **Count the parameters in attention and in the feed-forward** and report the ratio.

- 3 **Compare pre-norm against post-norm** at twelve layers, with and without warmup.
- 4 **Demonstrate permutation equivariance:** shuffle the input and show the outputs shuffle identically with positional encoding removed.
- 5 Implement sinusoidal, learned, and rotary positional encodings and **compare extrapolation beyond the trained length**.
- 6 **Visualise attention patterns per head** and try to characterise what each attends to.
- 7 **Prune heads one at a time** and measure the loss increase for each. Report how many are removable.
- 8 Convert a decoder to an encoder by removing the causal mask and observe what breaks.
- 9 **Fit a scaling law** on your own small models across three sizes and extrapolate to a fourth. Then train it and check.
- 10 Take a metric that is discontinuous and one that is smooth, and **plot the same models under both**.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the four components of a transformer block and the job of each.
- 2 Which component holds most of the parameters?
- 3 What is the difference between pre-norm and post-norm, and which wins?
- 4 Why use multiple heads rather than one?
- 5 Why is positional encoding necessary at all?
- 6 Compare five positional schemes on extrapolation.
- 7 Distinguish encoder, decoder, and encoder–decoder, and say what the difference actually is.
- 8 State the scaling laws and what Chinchilla corrected.
- 9 Give four reasons the transformer spread across domains.